Service Requests Portal — Next Steps

Written: 2026-08-07 **Prototype owner:** Valya (Citizen Developers program) **Decision requested from:** the Customer Interactions team

---

What this app is, and where it came from

The Service Requests Portal was built in the **Citizen Developers program**, in which employees who are not developers build working solutions to problems they own. It is a working prototype: it runs, it holds data, and it settles the product questions — what the workflow is, what each screen has to do, which rules are load-bearing, and where the sharp edges are.

Every app that comes out of the program ends in one of three ways:

Outcome	Means
Retired	The solution was wrong, or the problem went away. Nothing carries forward.
Hardened in place	The prototype itself is taken on, brought up to company standards, and deployed as-is.
Rebuild	Developers re-create the application from the prototype, on the organization's own stack and standards.

This is a REBUILD

Not retired, and not hardened in place. The reasoning, briefly:

Why not retired. The problem is real and the solution works. Before the portal there was no system of record: defect reports lived in email threads and spreadsheets, duplicates were near-impossible to spot, enhancement requests arrived incomplete, and reps had no way to see whether something was already known — so they filed it again. Duplicate intake was the single largest source of wasted triage effort. The portal closes that loop, and it has since grown to cover report and dashboard requests as well, which arrived through the same channel and had the same problem.

Why not hardened in place. The prototype makes a set of choices that are correct for a prototype and wrong for a production system, and they are not cosmetic:

- Attachments are stored where the hosting made easy, in a **public** bucket. They are screenshots that may carry customer policy and account data. They need to sit behind authorization.
- The schema **self-migrates on boot** in production. Convenient while moving fast; unreviewable and irreversible as a deploy strategy.
- Timestamps are ISO strings in `TEXT` columns, and `updated_at` doubles as the optimistic-concurrency token compared as a string. That needs a proper version column, and the conversion is cross-cutting rather than a type change.
- Rate limiting and ticket presence are in-memory, so the app cannot run more than one instance.

- Identity is a local username/password login. The target is SSO against Active Directory. Every seam for it exists; nothing is connected.
- The hosting shape (a static host that cannot carry WebSocket upgrades) forced a cross-origin socket and a signed-token auth path that exists for no product reason and should be deleted rather than reproduced.

None of these is hard to do correctly. All of them are easier to do correctly from the start than to retrofit. That is the case for a rebuild rather than a hardening pass.

What a rebuild inherits. The prototype is not thrown away — it is the specification. `DEVELOPER_HANDOFF.md` carries every decision and the reason for it, an acceptance checklist of the behaviors that are load-bearing, and a full inventory of what to delete rather than reproduce. `USER_MANUAL.md` documents every feature as it works today. The screenshots are current as of this date and are re-takeable with one command.

What is needed now

1. Prioritization by the Customer Interactions team

The rebuild needs to be **prioritized by the Customer Interactions team**. It is not a candidate for spare capacity: it replaces a process that is currently running on this prototype, and the prototype is deployed on external hosting with a shared test database, which is not where it should stay.

Two things worth knowing when it is sized:

- **The product questions are settled.** The workflow, the field lists, the status vocabulary, the access model and the screen behaviors are all decided, built, used and documented. A rebuild team is not doing discovery.
- **Two decisions are genuinely open** and belong to the engineering team, not to the product owner: the **database engine**, and the **hosting shape**. Both are laid out with their consequences in the handoff — the dialect-sensitive surface is small and fully inventoried, and the reverse-proxy requirements are listed explicitly.

2. Valya to remain product owner

Valya would like to remain product owner of this application going forward, if that is an option.

The reason it is worth considering rather than just accommodating: the decisions in this portal are not generic. Whether a report request is private, why a redirect moves a ticket rather than copying it, why a cleanup task is a flag rather than a request type, why the approval field is a typed name rather than a user id, what the nine report-request statuses mean and why three of them replaced five — every one of those came out of the work the Customer Interactions team actually does, and several were corrected mid-build because the first answer was wrong in a way only somebody doing the job would notice. Continuity of that judgment is worth more to a rebuild than any document, including this one.

Handover state

Everything below is true as of 2026-08-07.

Branch	`main`, clean and pushed. `main` is the only working branch.
Feature work	Complete. Nothing is in flight and no decision is outstanding.
Verification	378 server tests, client lint and production build clean, and the browser harness green across seven scripts at 1500/820/390px in both themes.
Documentation	This document, the developer handoff, and the user manual.
Screenshots	Current, and regenerated by one command (`node scripts/capture-screenshots.mjs` from `client/`).
Data	The shared database holds a purpose-built demonstration set — 39 requests across all four kinds of work, three applications and every status. It is all test data.
Test accounts	Documented in the user manual and the handoff, so the app can be opened and driven without asking anyone.

The three things to do first

1. **Read the handoff's acceptance checklist** before writing any code. It is a list of behaviors that each either encode a domain rule or fix a bug that was actually hit. Breaking one is a regression, not a design difference.
2. **Take the two open decisions** — database engine and hosting shape — early. Everything else in the rebuild is downstream of them.
3. **Rotate the credentials.** The shared test password and the AI provider key currently live in a gitignored `.env` on the prototype. They should be in a secret store before anything real runs.

What is explicitly NOT being asked for

- **Not** a like-for-like port. Several parts of the prototype exist only because of its hosting and should be deleted; the handoff names all of them.
- **Not** the EasyVista integration as built. The payload shape, endpoint path and response parsing are still unconfirmed on the EasyVista side, and there is a known EasyVista-side defect (it overwrites the `Description` field with empty form results). That conversation needs to happen with the EasyVista team before anyone rebuilds against the current guesses.
- **Not** the AI semantic search as a requirement. It is optional and self-disabling — the portal is fully functional without it — so a delayed egress approval cannot block a deployment. It is worth keeping, and the `AI_PROVIDER=anthropic` setting keeps embeddings in-process so ticket text never leaves the network.